

mcp-ratchet — Tool Qualification Context & TQL Determination

Establishing the operational scenario, deciding whether the tool needs qualification at all, and determining its Tool Qualification Level before any qualification work begins.

Doc ID: MR-TQ-001 Rev: 2026-08-29 Tool under study: mcp-ratchet, drift-detection function

Result: Candidate TQL-5, scope bounded to the deterministic function Read Order: 1 of 5

This study works one question end to end: if a DO-178C program wants to lean on an AI assistant during verification, and wants a drift monitor watching that assistant's interface, what does DO-330 require of the drift monitor? This first document does not qualify anything. It fixes the scenario, applies the DO-178C 12.2 qualification trigger, and lands on a candidate Tool Qualification Level so the later documents have a target to work against. Everything here is a proposal for a human with compliance authority to accept, revise, or reject.

§1 What this document is, and is not

DO-330 (*Software Tool Qualification Considerations*) is the standalone RTCA document that DO-178C section 12.2 points to for *how* to qualify a tool once you have decided one is needed. The full text is paywalled. This study does not reproduce it and does not assert specific per-TQL objective counts from its tables. TQL-determination criteria below are cited from named secondary sources and cross-checked against the sibling [AAS-TQ-002](#) study, which corrected its own earlier misread of the same table.

- In scope:** the operational scenario, the 12.2.1 qualification trigger, the 12.2.2 criteria, a candidate TQL, and an explicit scope boundary around which part of mcp-ratchet is being considered.
- Not in scope here:** the Tool Operational Requirements themselves (MR-TQ-002), the qualification plan (MR-TQ-003), verification (MR-TQ-004), and the final verdict (MR-TQ-005).
- Never in scope:** qualifying the underlying AI assistant. That is a separate, harder effort. This study is about the *monitor*, not the thing it monitors.

§2 The operational scenario

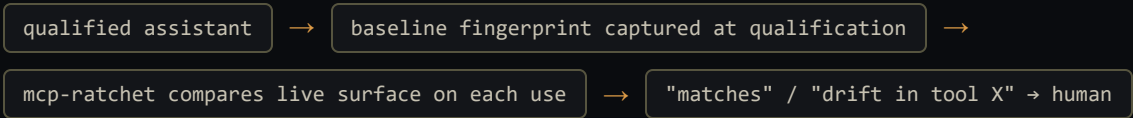
A DO-178C development or verification activity uses one or more AI assistants exposed as MCP servers. A concrete example: an LLM-backed agent that, given a low-level requirement and its implementing source function, proposes whether the two correspond and flags requirements it judges untestable, for a human reviewer to confirm or overturn. Suppose the program elects to *credit* this assistant — that is, to use its output to reduce the manual effort in a DO-178C process rather than treating it as a convenience with no compliance weight. Under 12.2.1 that assistant now requires qualification.

The MCP-specific hazard — An MCP server advertises its capabilities as a set of *tool definitions*: each tool's name, natural-language description, and input schema. A server can change these definitions at any time, including after a human has reviewed and approved them. This is a documented attack and failure class (informally, a "rug pull"). If the credited assistant's advertised interface changes after qualification, the tool in use is no longer the tool that was qualified — and nothing in a normal review workflow surfaces that.

The program's response is to place a **configuration-integrity control** on the credited assistant: capture an exact fingerprint of its tool surface at qualification time, and thereafter verify on every use that the deployed surface still matches. mcp-ratchet is the tool proposed for that control.

§3 mcp-ratchet's role and its output claims

mcp-ratchet has several functions. This study considers **only its runtime drift-detection function**: it records a canonical fingerprint of every tool a server advertises (name, description, input schema) at a chosen baseline moment, and on later runs reports, per tool, whether the current definition matches that baseline or has drifted. The fingerprint-and-compare logic is fully deterministic — the same inputs always produce the same verdict.



The output claim the program would rely on is narrow and testable:

Reliance claim — When mcp-ratchet reports "no drift" for a server against a stored baseline, every tool definition that server currently advertises is byte-for-byte equivalent, after canonicalization, to the definition captured in that baseline.

Its contrapositive is the safety-relevant half: a real change to any advertised tool definition must produce a "drift" report, not a "no drift" report. A false "no drift" is the failure that matters.

§4 Does mcp-ratchet require qualification?

DO-178C 12.2.1 asks two things: is a tool's output used to eliminate, reduce, or automate a DO-178C process, and could the tool then either insert an error into the airborne software or fail to detect an existing error. Applied here:

QUESTION	FOR MCP-RATCHET'S DRIFT-DETECTION FUNCTION
Does its output automate or reduce a DO-178C process?	Yes. If the program credits the AI assistant, it must show the assistant stays in its qualified configuration. Crediting mcp-ratchet's "no drift" output to make that showing automates part of the configuration-management / integrity-verification activity for a qualified tool.
Could it insert an error into the airborne software?	No. Its output never becomes part of, or directly shapes, the airborne software. It reports on another tool's interface.
Could it fail to detect an error?	Yes. A false "no drift" would let a changed, no-longer-qualified assistant configuration keep receiving qualification credit. The error — an invalid qualification basis — goes undetected.

TRIGGER MET

The drift-detection function is used to reduce a DO-178C process and could fail to detect an error. It requires qualification. The remaining question is at what level.

One honest caveat: whether the trigger fires at all depends entirely on the program *choosing* to credit the AI assistant. A program that uses the assistant purely as an unremarked convenience, with every output independently re-derived by a human, credits nothing and qualifies nothing — neither the assistant nor mcp-ratchet. This study assumes the program does credit the assistant, because that is the only case where any of this matters.

§5 Qualification criteria and candidate TQL

DO-178C 12.2.2 sorts qualifying tools into three criteria; the criterion plus the software level of the airborne code sets the TQL. The mapping below is reproduced from named secondary sources (Rapita Systems; LegalClarity) and matches the corrected table in [AAS-TQ-002 §1](#) . The exact numbered objective list per TQL is DO-330 table content and is not reproduced.

CRITERIA	TQL @ A	TQL @ B	TQL @ C	TQL @ D
1 — output could insert an error into the airborne software	TQL-1	TQL-2	TQL-3	TQL-4
2 — could fail to detect an error, <i>and</i> its output justifies eliminating or reducing a process other than the one it automates	TQL-4	TQL-4	TQL-5	TQL-5
3 — could fail to detect an error	TQL-5	TQL-5	TQL-5	TQL-5

Not Criteria 1. §4 already established mcp-ratchet's output does not enter the airborne software.

Criteria 2 or Criteria 3 turns on a single question: does mcp-ratchet's output justify eliminating or reducing *a different process*, or is it an additional check layered on top of processes that stay in place?

HOW THE PROGRAM USES MCP-RATCHET	CRITERION	TQL (LEVEL A-D)
Replaces a manual re-review of the assistant's interface that the program would otherwise perform on each use	2	TQL-4 / TQL-4 / TQL-5 / TQL-5
Added <i>on top of</i> existing configuration management; a human still independently reviews the assistant's interface on the program's normal CM cadence	3	TQL-5 at every level

CANDIDATE: CRITERIA 3, TQL-5

This study proposes the program use mcp-ratchet as an *additional* automated control, not a replacement for any manual process. A human still reviews every drift alert, and the assistant's interface is still independently re-reviewed on the program's normal configuration-management schedule. Under that usage mcp-ratchet stays at Criteria 3, which is TQL-5 regardless of the airborne software level. MR-TQ-003 onward is written to the TQL-5 target.

IF THE PROGRAM WANTS THE CRITERIA 2 USAGE INSTEAD

Using mcp-ratchet to eliminate a manual interface re-review moves it to Criteria 2, which is TQL-4 at Levels A and B — a materially heavier qualification. That path is not worked in this study. It would need its own plan and sign-off from whoever holds compliance authority on the program.

§6 Scope boundary — the deterministic function only

mcp-ratchet also runs a prompt-injection check on tool descriptions. That check makes a live call to a large language model and asks it to judge whether a description contains text aimed at manipulating the agent reading it. It is useful, but it is **not deterministic**: the same description can draw a different verdict on different runs, and the model's behavior can change under the tool author's feet.

THE INJECTION CHECK IS EXCLUDED FROM QUALIFICATION SCOPE

A tool's qualified function has to be verifiable against fixed Tool Operational Requirements. A check whose output is not reproducible cannot meet that bar. This study qualifies the fingerprint-and-compare drift-detection function and nothing else. The injection check may still run operationally — it just carries no qualification credit, and MR-TQ-002's requirements will say so explicitly.

This is the same discipline the sibling `AAS-TQ-002` used when it bounded its scope to `gcov` and explicitly left `gcc` out: a qualification argument only holds for the exact function you wrote requirements for and verified, never for "the tool" as a loose whole.

§7 What a configuration-integrity control does and does not buy

It is worth being blunt about the ceiling on this work, so the later documents are not oversold.

A QUALIFIED DRIFT MONITOR GIVES YOU	IT DOES NOT GIVE YOU
Assurance that the credited assistant's advertised interface has not changed since it was qualified	Any assurance about whether the assistant's <i>judgments</i> are correct — that is the underlying tool's own qualification problem
A defined, testable trigger for re-qualification when the interface does change	Protection against a change in the model's behavior that leaves the advertised interface untouched
A deterministic, reproducible check that fits inside a normal TQL-5 effort	A reason to skip independent human review of the assistant's output

THE HONEST FRAMING

This study does not show that AI tools can be safely qualified for DO-178C credit. It shows that *one specific, deterministic guardrail* around such a tool can itself be qualified cheaply, and that the guardrail's value is bounded to holding the edges of an existing qualification — not to establishing one.

§8 Carried forward to MR-TQ-002

- Write Tool Operational Requirements for the drift-detection function: what "match" and "drift" mean precisely, what canonicalization is applied before comparison, and the required behavior when a baseline is missing, malformed, or partially readable.
- State explicitly, as a requirement, that the injection check carries no qualification credit.
- Define the operational environment the tool must be qualified in — OS, Python version, how the baseline file is stored and protected, who may update it.
- Identify the one genuinely AI-shaped verification problem that remains even at TQL-5: showing the fingerprint captures *everything* a model could act on in a tool definition, not just the fields mcp-ratchet currently reads.